Reinforcement Learning and Dynamic Optimization

Professor: Thrasyvoulos Spyropoulos

Card Counting Black Jack: Phase 2



ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

Name:	Konstantinos Bouros
ID:	2021030091
Name:	Pavlos Zounits
ID:	2021030005

Electrical and Computer Engineering
Greece

02.07.2025

Introduction

In this exercise, we continue the work made in the previous phase. We add the actions double, split and we implement 3/2 blackjack (task 1). Then we add the possibility to choose the betting amount in each game (task 2).

Task 1

For the first task, we add the possibility to double only when we have 2 cards (initial hand). If this cards have the same value, we allow splitting (unlimited splitting allowed). Now if we manage to get a straight blackjack (with only 2 cards) and win, then the reward is multiplied by 1.5.

It is worth mentioning that we included the boolean variables `splittable` and `double_flag` in the state representation to prevent the selection of illegal actions and to avoid combining learning across game states with differing action spaces during training and testing. We maintained the simple high/low counting method with a threshold of 3, and trained the agent using tabular Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha(|Q(s,a)|)\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

with the following parameters:

- $\gamma = 1$
- $\epsilon = \max\{0.15, 0.999^i\}$, ($i$ = current episode)
- $\omega = 0.8$
- $\alpha(|Q(s,a)|) = \frac{1}{|Q(s,a)|^\omega}$ ($|Q(s,a)|$ is the number of times $Q(s,a)$ being chosen)
- Training episodes: 10^7
- Testing episodes: 10^7

The idea for the decreasing learning rate α came from the following paper.

```
Results for task 1:
Profit per episode: 0.01268085
```

Figure 1: Profit

The positive reward, even if we do not control the betting amount yet (1€ bets), can be explained by the inclusion of the boolean variables in the state space and the fact that we refill and reshuffle the card deck quite "late" (only when 10 or fewer cards are remaining, and a new game starts), giving us much information about what card value to expect.

Now we present the produced policies for all the counting states.

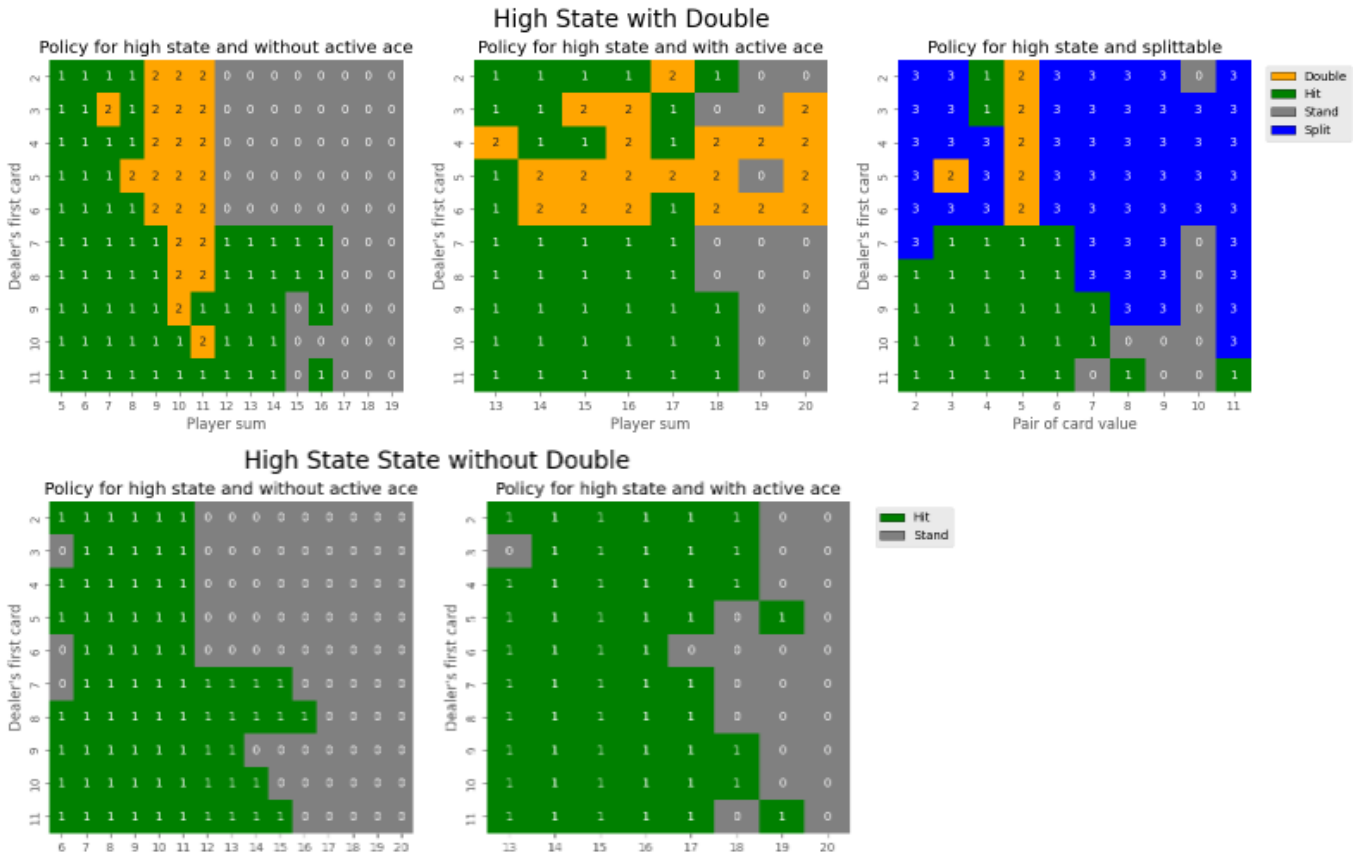


Figure 2: Produced policy in high state

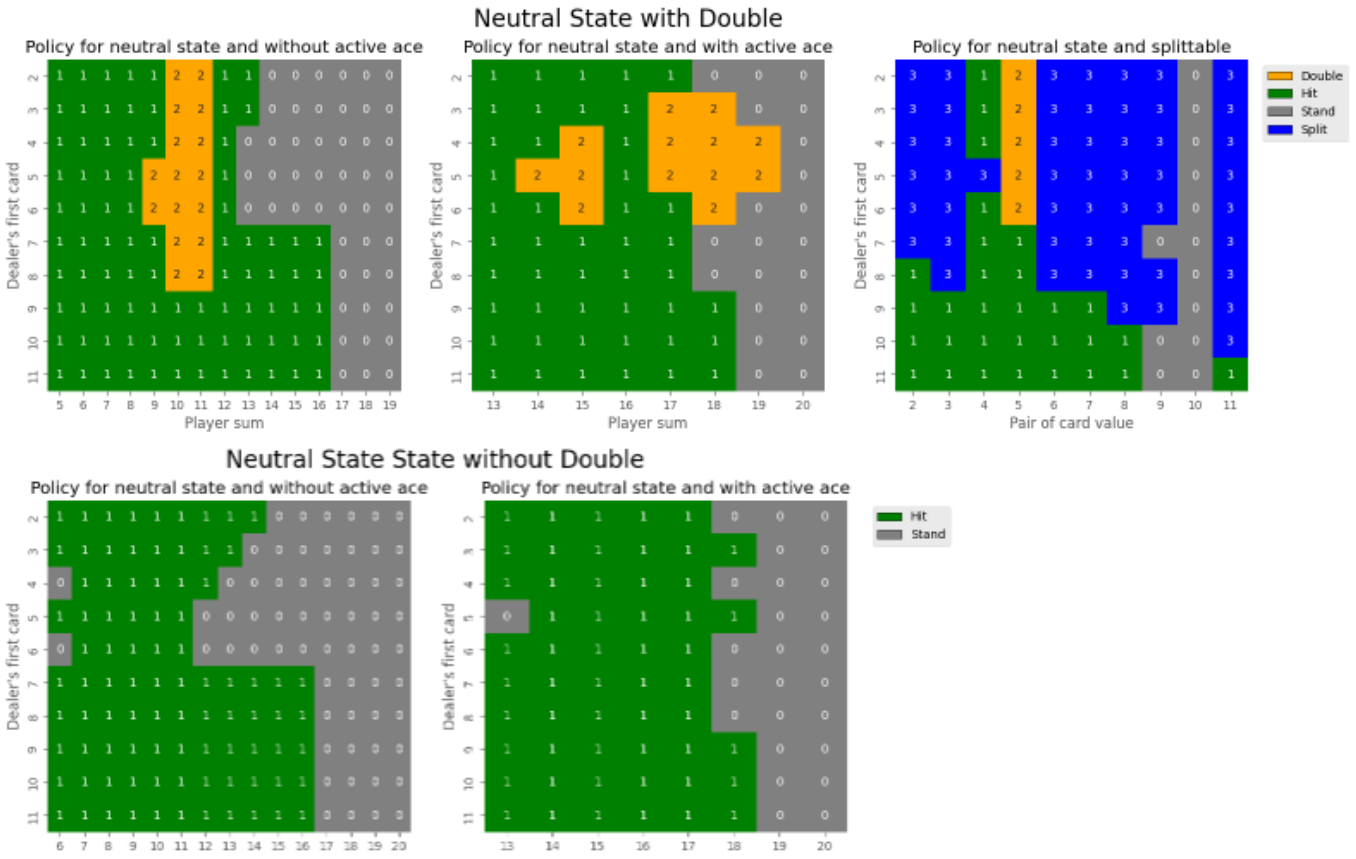


Figure 3: Produced policy in neutral state



Figure 4: Produced policy in low state

Generally, we can observe that whenever we can split, we prefer to do so. The only exceptions are when we have 10 value cards, in which case the sum is equal to 20, where it is better to stand, and when we have fives, where it is better to hit or double, depending on the dealer's card. When we cannot split and do not have an active ace, we prefer to double with a sum of 10 or 11 and a low-value dealer's card. When we have an active ace, we double for a broader range of sum, again with a low-value dealer's card. However, we seem to double less as the counting state decreases because the expected card value also decreases.

Task 2

Having achieved a positive reward, we continued and figured out a way to maximize profits by selecting a betting amount (with `max_bet` = 40€) in the beginning of each game, having as information only the counting state.

First approach to solve the problem

Firstly, we tried to implement the DQN algorithm. We switched to a true count system of size 7 ($[-7, -6, \ldots, -1, 0, 1, \ldots, 7]$), which gives us a better understanding of the cards remaining in the deck.

```
class DQNetwork(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DQNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.LayerNorm(256),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.LayerNorm(128),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.LayerNorm(64),
            nn.Linear(64, output_dim)
        )
```

Figure 5: Neural network used

In order to find the optimal policy, we trained the betting policy and the game policy (with the betting amount as a state variable) using two instances of DQNetwork at the same time. We set up the following neural network and trained it with the following parameters:

- Training episodes: 10^5 (decreased due to excessive time)
- Testing episodes: $5 \cdot 10^5$
- Learning rate: 0.0002
- $\gamma = 1$
- $\epsilon = \max\{0.15, 0.9999^i\}$
- Batch size: 128
- Memory size: 10^4
- Target update freq: 700

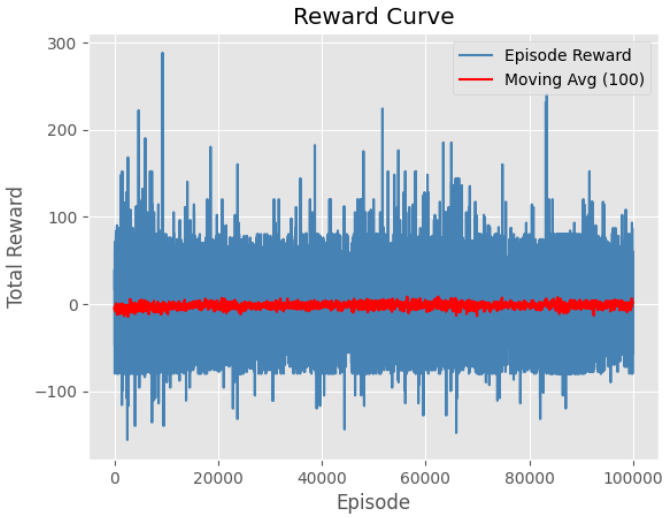


Figure 6: Reward with true count

```
Results for task 2 with DQN (true count):
Profit per episode: -0.694373
```

Figure 7: Profit with true count

It is worth mentioning that this negative result would be even lower (approximately -2€ per episode) if we did not use a warmup game policy produced by tabular Q-Learning under the same counting system.
Then, we included all occurrences of each card, which is the most complete (and most complex) counting method. This time, we do not start with a warmup policy as it cannot be easily trained with Q-tabular learning, taking into account the extremely large state space.

```
Results for task 2 with DQN (complete count):
Profit per episode: -1.738122
```

Figure 8: Profit with complete count

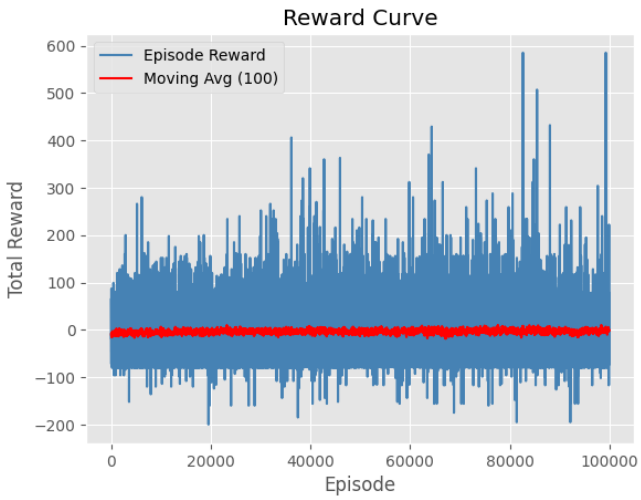


Figure 9: Reward with complete count

This gives us a worse profit, even though we used a more sophisticated counting method. This is likely due to the absence of warm training in this scenario. Additionally, it is worth that the negative profit of both DQN options is likely due to an insufficient amount of training.

Approach with tabular Q Learning

After having failed with DQN, we came back to tabular Q Learning but with the true count method mentioned before ($[-7, -6, \ldots, -1, 0, 1, \ldots, 7]$). We used the same parameters as task 1, except for the following changes.

- $\epsilon = \max\{0.15, 0.99995^i\}$
- Training episodes: 10^7
- Testing episodes: 10^7

We first train the playing policy with 1€ bets only and get a policy similar to the one in the previous task. Then we train only the betting policy based on the learned playing policy and get the following results:

```
---- Betting Policy: ----
Bet for true count -7: 1
Bet for true count -6: 1
Bet for true count -5: 1
Bet for true count -4: 1
Bet for true count -3: 1
Bet for true count -2: 1
Bet for true count -1: 1
Bet for true count 0: 20
Bet for true count 1: 28
Bet for true count 2: 30
Bet for true count 3: 40
Bet for true count 4: 38
Bet for true count 5: 34
Bet for true count 6: 35
Bet for true count 7: 40
```

Figure 10: Betting policy

The betting policy informs us to bet heavily on higher-counting states and bet minimal amounts on lower-counting states. This is a well-known rule among blackjack players, and can be explained by the fact that the dealer is more likely to bust during high-counting states, while we can achieve higher sums (not over 21) with higher probability.

```
Results for task 2 with tabular Q-learning:
Profit per episode: 0.4275433
```

Figure 11: Profit

This gives us a profit of 0.42€ per episode, which is a realistic profit to expect when counting in blackjack.

It is worth noting that if we didn't separate the learning of the two policies (betting and playing), then the profit would be much lower than what we achieve (approximately 0.06€ per episode). This can be explained by the fact that during the early episodes of training, our bad playing policy leads to penalizing higher betting strategies, leading to non-minimal bets only for the counting states of 6 and 7.

Same experiment with 4 decks

Finally, we ran the same experiment but this time with 4 decks instead of 1 which gave us the following betting policy (again, the playing policy is similar).

```
---- Betting Policy: ----
Bet for true count -7: 1
Bet for true count -6: 1
Bet for true count -5: 1
Bet for true count -4: 1
Bet for true count -3: 1
Bet for true count -2: 22
Bet for true count -1: 1
Bet for true count 0: 1
Bet for true count 1: 16
Bet for true count 2: 29
Bet for true count 3: 36
Bet for true count 4: 29
Bet for true count 5: 24
Bet for true count 6: 29
Bet for true count 7: 40
```

Figure 12: Betting policy with 4 decks

The produced betting policy is very similar to the one with only 1 deck, (except for true count of -2, which is likely a statistical mistake) which makes sense because there is no reason to change our strategy because of higher number of cards.

```
Results for task 2 with tabular Q-learning:
Profit per episode: 0.17013115
```

Figure 13: Profit with 4 decks

However, the profit per episode is lower than what we achieved before. This is due to the fact that with more decks we get less counting information, as the distribution of drawn cards tends to become uniform (like an infinite deck).